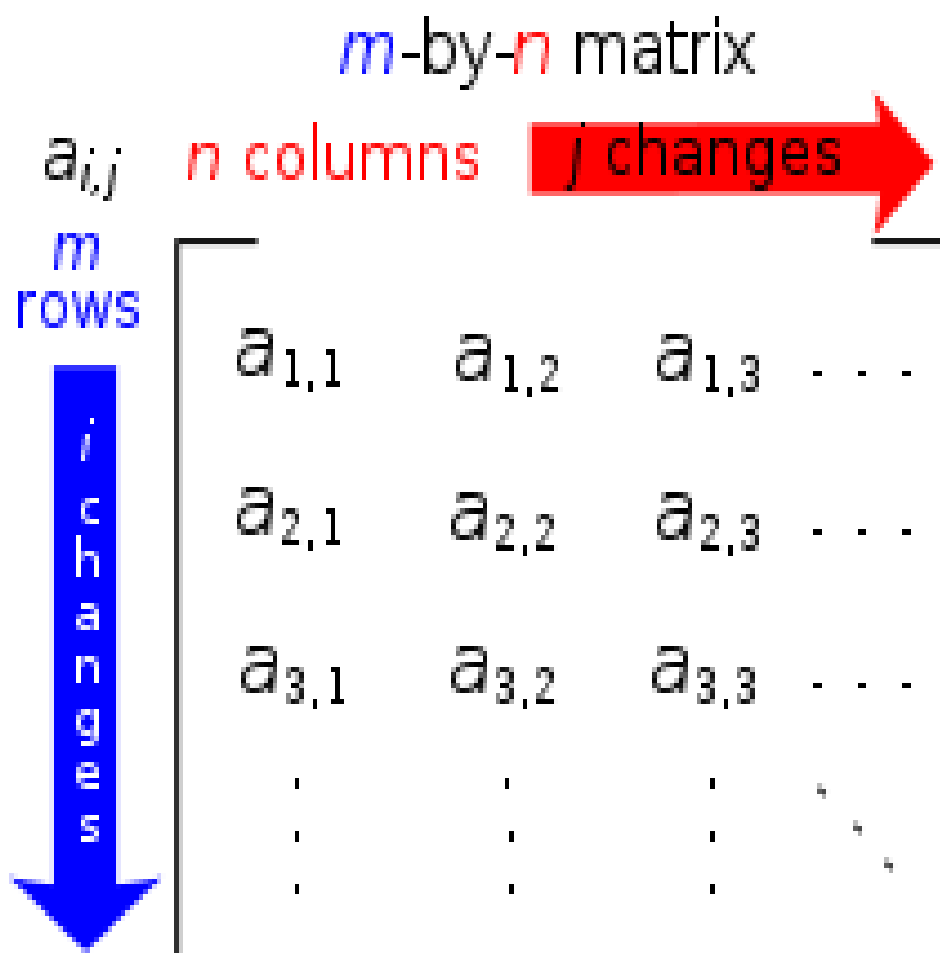


Matrices ----- Notation and Operations

- A vector is a one dimensional array
- A Matrix is a two dimensionall array: a rectangle of numbers

Matrix notation

Matrix Notation is a notation system that allows succinct representation of complex operations, such as a change of basis.



- **Matlab** is based on Matrix Notation.
- **Python**: similar functionality by using **numpy**

Recall that a **vector** can be represented as a one dimensional array of numbers. A **matrix** is a two dimensional rectangle of numbers. A matrix consists of rows, indexed from the top to the bottom and of columns, indexed from the left to the right. As is described in the figure.

A matrix with n rows and m columns is said to be an " m by n matrix". In numpy we will say that the **shape** of the matrix is (m, n) . We will also use the LaTeX notation $M_{m \times n}$ to indicate that M is an $m \times n$ matrix.

Transposing a Matrix

At times it is useful to switch the rows and column dimensions of matrices. Consider the matrix

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \quad (1)$$

The transpose of A is

$$A^T = \begin{bmatrix} a_{11} & a_{21} & a_{31} \\ a_{12} & a_{22} & a_{32} \end{bmatrix} \quad (2)$$

```
In [1]: import numpy as np
# The .reshape command reorganized the elements of a matrix into a new shape
A = np.array(list(range(6)))
print('A=',A)
B=A.reshape(2,3)
print("B is a 2X3 matrix:\n",B)
print("the shape of B is:",B.shape)
print("The transpose of B is\n",B.T)
print("the shape of B.T is:",B.T.shape)
```

```
A= [0 1 2 3 4 5]
B is a 2X3 matrix:
[[0 1 2]
 [3 4 5]]
the shape of B is: (2, 3)
The transpose of B is
[[0 3]
 [1 4]
 [2 5]]
the shape of B.T is: (3, 2)
```

Vectors as matrices.

When using matrix notation, vectors can be represented as either [row](#) or [column vectors](#). In a matrix context, a vector $\vec{v}$ is denoted by a bold-face letter. $\mathbf{v}$ for a column vector and $\mathbf{v}^T$ for row vector:

- By default a vector is represented as a **column vector** which is a matrix consisting of a single column:

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_d \end{bmatrix} \quad (3)$$

- If $\vec{v}$ is a column vector then the **transpose** of $\vec{v}$, denoted by $\vec{v}^\top$ is a **row vector** which is a matrix consists of a single row:

$$\mathbf{v}^\top = [v_1 \quad v_2 \quad \cdots \quad v_d] \quad (4)$$

A vector as a matrix

Row and Column vectors can be thought of as matrices.

- The column vector $\mathbf{v}$ is a $d \times 1$ matrix.
- The row vector $\mathbf{v}^\top$ is a $1 \times d$ matrix.

A matrix as a collection of vectors

Matrices can be represented as a collection of vectors. For example, consider the 2×3

$$\text{matrix } \mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}$$

We can represent $\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}$ as vectors in one of two ways:

- As a row of column vectors:

$$\mathbf{A} = [\mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3]$$

where

$$\mathbf{c}_1 = \begin{bmatrix} a_{11} \\ a_{21} \end{bmatrix}, \mathbf{c}_2 = \begin{bmatrix} a_{12} \\ a_{22} \end{bmatrix}, \mathbf{c}_3 = \begin{bmatrix} a_{13} \\ a_{23} \end{bmatrix}$$

- Or as a column of row vectors: $\mathbf{A} = \begin{bmatrix} \mathbf{r}_1 \\ \mathbf{r}_2 \end{bmatrix}$

where $\mathbf{r}_1 = [a_{11}, a_{12}, a_{13}]$, $\mathbf{r}_2 = [a_{21}, a_{22}, a_{23}]$,

```
In [2]: A=np.array(list(range(6))).reshape(2,3)
print('A=\n',A)
```

```
A=
[[0 1 2]
 [3 4 5]]
```

```
In [3]: print("Splitting A into columns:")
Columns=np.split(A,3,axis=1)
for i in range(len(Columns)):
    print('column %d'%i)
    print(Columns[i])
```

```
Splitting A into columns:
column 0
[[0]
 [3]]
column 1
[[1]
 [4]]
column 2
[[2]
 [5]]
```

```
In [4]: A_recon=np.concatenate(Columns,axis=1)
print('reconstructing the matrix from the columns:')
print(A_recon)
print('Checking that the reconstruction is equal to the original')
print(A_recon==A)
```

```
reconstructing the matrix from the columns:
[[0 1 2]
 [3 4 5]]
Checking that the reconstruction is equal to the original
[[ True  True  True]
 [ True  True  True]]
```

```
In [5]: print("Splitting A into rows:")
Rows=np.split(A,2,axis=0)

for i in range(len(Rows)):
    print('row %d'%i)
    print(Rows[i])
```

```
Splitting A into rows:
row 0
[[0 1 2]]
row 1
[[3 4 5]]
```

```
In [7]: A_recon=np.concatenate(Rows,axis=0)

print('reconstructing the matrix from the rows:')
print(A_recon)
print('Checking that the reconstruction is equal to the original')
print(A_recon==A) #equality checked element by element
```

```
reconstructing the matrix from the rows:
[[0 1 2]
 [3 4 5]]
Checking that the reconstruction is equal to the original
[[ True  True  True]
 [ True  True  True]]
```

Numpy functions

Beyond the commands `reshape`, `split` and `concatenate` numpy has a rich set of functions to manipulate arrays, for a complete list see [Numpy Array Manipulation routines](#)

Matrix - scalar operations

You can add/subtract multiply/divide a scalar from a matrix

Adding a scalar value to a matrix

Let $A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$. Here is how we would add the scalar 3 to A :

$$A + 3 = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} + 3 = \begin{bmatrix} a_{11} + 3 & a_{12} + 3 \\ a_{21} + 3 & a_{22} + 3 \end{bmatrix} \quad (5)$$

Product of a scalar and a matrix

Multiplication by a scalar is similar

$$3 \times A = 3 \times \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} = \begin{bmatrix} 3a_{11} & 3a_{12} \\ 3a_{21} & 3a_{22} \end{bmatrix} \quad (6)$$

```
In [13]: # Some examples of matrix-scalar operations using numpy
print('A=\n',A)
print('A+3=3+A=\n',A+3) # addition

print('A*3=\n',A*3) # product

print('A/2=\n',A/2) # integer division
print('A/2.=\n',A/2.) # floating point division
```

```
A=
[[0 1 2]
 [3 4 5]]
A+3=3+A=
[[3 4 5]
 [6 7 8]]
A*3=
[[ 0  3  6]
 [ 9 12 15]]
A/2=
[[0.  0.5 1. ]
 [1.5 2.  2.5]]
A/2.=
[[0.  0.5 1. ]
 [1.5 2.  2.5]]
```

Adding and subtracting two matrices

Let $A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$ and $B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$. To compute $A - B$, subtract each element of B from the corresponding element of A :

$$\begin{aligned} A - B &= \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} - \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} \\ &= \begin{bmatrix} a_{11} - b_{11} & a_{12} - b_{12} \\ a_{21} - b_{21} & a_{22} - b_{22} \end{bmatrix} \end{aligned}$$

Addition works exactly the same way:

$$\begin{aligned} A + B &= \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} + \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} \\ &= \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{12} \\ a_{21} + b_{21} & a_{22} + b_{22} \end{bmatrix} \end{aligned}$$

An important point to know about matrix addition and subtraction is that it is only defined when A and B have the same shape. Here, both are 2×2 . Since operations are performed element by element, these two matrices must be conformable- and for addition and subtraction that means they must have the same numbers of rows and columns. I like to be explicit about the dimensions of matrices for checking conformability as I write the equations, so write

$$A_{2 \times 2} + B_{2 \times 2} = \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{12} \\ a_{21} + b_{21} & a_{22} + b_{22} \end{bmatrix}_{2 \times 2}$$

Notice that the result of a matrix addition or subtraction operation is always of the same dimension as the two operands.

Let's define another matrix, B , that is also 2×2 and add it to A :

```
In [8]: B = np.random.randn(2,2)
print(B)
```

```
[[-0.74860455 -0.80419417]
 [ 0.11639833 -0.72544633]]
```

```
In [9]: print(A.shape,B.shape)
result = A + B
```

```
(2, 3) (2, 2)
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[9], line 2
      1 print(A.shape,B.shape)
----> 2 result = A + B

ValueError: operands could not be broadcast together with shapes (2,3) (2,2)
```

Matrix-Matrix products

The dot product of two vectors

- Recall that a vector is just a skinny matrix.
- Consider the dot product $(1, 2, 3) \cdot (1, 1, 0) = 1 \times 1 + 2 \times 1 + 3 \times 0 = 3$.

Conventions of dot product in matrix notation:

- The first vector is a row vector and the second vector is a column vector.
- In matrix notation there is no dot ($\cdot$) between the two vectors

$$[1, 2, 3] \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} = 1 \times 1 + 2 \times 1 + 3 \times 0 = 3$$

```
In [21]: A=np.array([1,2,3])
B=np.array([[1,1,0]]).T
print('A=',A)
print('B=',B)
A@B
```

```
A= [1 2 3]
B= [[1]
    [1]
    [0]]
```

```
Out[21]: array([3])
```

The dot product of a matrix and a vector

To multiply the matrix $\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}$ by the column vector $\mathbf{c} = \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix}$

We think of $\mathbf{A}$ as consisting of two row vectors: $\mathbf{A} = \begin{bmatrix} \mathbf{r}_1 \\ \mathbf{r}_2 \end{bmatrix}$

where $\mathbf{r}_1 = [a_{11}, a_{12}, a_{13}]$, $\mathbf{r}_2 = [a_{21}, a_{22}, a_{23}]$,

and stack the results of dot products of $\mathbf{r}_1, \mathbf{r}_2$ with $\mathbf{c}$ to create a column vector of dimension 2:

$$\mathbf{Ac} = \begin{bmatrix} \mathbf{r}_1 \mathbf{c} \\ \mathbf{r}_2 \mathbf{c} \end{bmatrix} = \begin{bmatrix} a_{11}c_1 + a_{12}c_2 + a_{13}c_3 \\ a_{21}c_1 + a_{22}c_2 + a_{23}c_3 \end{bmatrix}$$

Dot product of two matrices

Multiplying a matrix and a column vector can be generalized to multiplying two matrices.

Consider a matrix $\mathbf{C}$ of size 2×3 and a matrix $\mathbf{A}$ of size 3×2

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix}, \mathbf{C} = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \end{bmatrix} \quad (7)$$

To compute $\mathbf{AC}$ we think of $\mathbf{A}$ as a column of row vectors: $\mathbf{A} = \begin{bmatrix} \mathbf{a}_1 \\ \mathbf{a}_2 \\ \mathbf{a}_3 \end{bmatrix}$

and of $\mathbf{C}$ as a row of column vectors: $\mathbf{C} = [\mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3]$

$\mathbf{AC}$ is the matrix generated from taking the dot product of each row vector in $\mathbf{A}$ with each column vector in $\mathbf{C}$

$$\begin{aligned} \mathbf{AC} &= \begin{bmatrix} \mathbf{a}_1 \\ \mathbf{a}_2 \\ \mathbf{a}_3 \end{bmatrix} [\mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3] = \begin{bmatrix} \mathbf{a}_1 \cdot \mathbf{c}_1 & \mathbf{a}_1 \cdot \mathbf{c}_2 & \mathbf{a}_1 \cdot \mathbf{c}_3 \\ \mathbf{a}_2 \cdot \mathbf{c}_1 & \mathbf{a}_2 \cdot \mathbf{c}_2 & \mathbf{a}_2 \cdot \mathbf{c}_3 \\ \mathbf{a}_3 \cdot \mathbf{c}_1 & \mathbf{a}_3 \cdot \mathbf{c}_2 & \mathbf{a}_3 \cdot \mathbf{c}_3 \end{bmatrix} = \\ &= \begin{bmatrix} a_{11}c_{11} + a_{12}c_{21} & a_{11}c_{12} + a_{12}c_{22} & a_{11}c_{13} + a_{12}c_{23} \\ a_{21}c_{11} + a_{22}c_{21} & a_{21}c_{12} + a_{22}c_{22} & a_{21}c_{13} + a_{22}c_{23} \\ a_{31}c_{11} + a_{32}c_{21} & a_{31}c_{12} + a_{32}c_{22} & a_{31}c_{13} + a_{32}c_{23} \end{bmatrix} \end{aligned}$$

For more information on the topic of matrix multiplication, see http://en.wikipedia.org/wiki/Matrix_multiplication.

```
In [24]: A=np.array([[1,2],[1,2],[0,1]])
B=np.array([[1,2,3],[1,2,3]])
print('A=',A)
print('B=',B)
A@B
```

```
A= [[1 2]
     [1 2]
     [0 1]]
B= [[1 2 3]
     [1 2 3]]
```

```
Out[24]: array([[3, 6, 9],
                [3, 6, 9],
                [1, 2, 3]])
```

```
In [25]: # Matrix - Vector product
A = np.arange(6).reshape((3,2))
C = np.array([-1,1])

print(A.shape)
print(C.shape)
print(np.dot(A,C.T))
```

```
(3, 2)
(2,)
[1 1 1]
```

```
In [27]: # Matrix - Matrix product

# Define the matrices A and C
A = np.arange(6).reshape((3,2))
C = np.random.randn(2,2)

print('A=\n',A)
print('C=\n',C)
```

```
A=
[[0 1]
 [2 3]
 [4 5]]
C=
[[-0.03810489 -0.36298179]
 [ 0.28274747 -0.70702213]]
```

We will use the numpy dot operator to perform the these multiplications. You can use it three ways to yield the same result:

```
In [28]: print('A.dot(C)=\n',A.dot(C))
print('np.dot(A,C)=\n',np.dot(A,C))
print('A@C=\n',A@C)
```

```
A.dot(C)=
[[ 0.28274747 -0.70702213]
 [ 0.77203263 -2.84702998]
 [ 1.2613178 -4.98703783]]
np.dot(A,C)=
[[ 0.28274747 -0.70702213]
 [ 0.77203263 -2.84702998]
 [ 1.2613178 -4.98703783]]
A@C=
[[ 0.28274747 -0.70702213]
 [ 0.77203263 -2.84702998]
 [ 1.2613178 -4.98703783]]
```

Conformity

Note that the number of columns in the first matrix has to be equal to the number of columns in the second matrix. Otherwise, the matrix product is not defined. When this condition holds we say that the two matrices **conform**.

Taking the product of two matrices that don't conform results in an exception:

```
In [29]: np.dot(C,A)
```

```

ValueError                                Traceback (most recent call last)
Cell In[29], line 1
----> 1 np.dot(C,A)

ValueError: shapes (2,2) and (3,2) not aligned: 2 (dim 1) != 3 (dim 0)

```

Orthonormal matrices and change of Basis

As was explained in the notebook: "Linear Algebra Review"

We say that the vectors $\vec{u}_1, \vec{u}_2, \dots, \vec{u}_d \in R^d$ form an **orthonormal basis** of R^d . If:

- **Normality:** $\vec{u}_1, \vec{u}_2, \dots, \vec{u}_d$ are unit vectors: $\forall 1 \leq i \leq d : \vec{u}_i \cdot \vec{u}_i = 1$
- **Orthogonality:** Every pair of vectors are orthogonal: $\forall 1 \leq i \neq j \leq d : \vec{u}_i \cdot \vec{u}_j = 0$

Orthonormal basis can be used to rotate the coordinate system:

- $\vec{v}$ is **represented** as a list of d dot products:

$$[\vec{v} \cdot \vec{u}_1, \vec{v} \cdot \vec{u}_2, \dots, \vec{v} \cdot \vec{u}_d]$$

- $\vec{v}$ is **reconstructed** by summing its projections on the basis vectors:

$$\vec{v} = (\vec{v} \cdot \vec{u}_1)\vec{u}_1 + (\vec{v} \cdot \vec{u}_2)\vec{u}_2 + \dots + (\vec{v} \cdot \vec{u}_d)\vec{u}_d$$

Orthonormal basis in matrix notation

To use matrix notation, we think of $\vec{u}_i$ as a row vector:

$$\mathbf{u}_i = [u_{i1}, u_{i2}, \dots, u_{id}],$$

We can stack the orthonormal vectors, one row per vector, to create an **orthonormal matrix**

$$\mathbf{U} = \begin{bmatrix} \mathbf{u}_1 \\ \mathbf{u}_2 \\ \vdots \\ \mathbf{u}_d \end{bmatrix} = \begin{bmatrix} u_{11}, u_{12}, \dots, u_{1d} \\ u_{21}, u_{22}, \dots, u_{2d} \\ \vdots \\ u_{d1}, u_{d2}, \dots, u_{dd} \end{bmatrix}$$

```

In [32]: import numpy as np

# Generate a random 4x4 matrix with entries from a normal distribution
A = np.random.randn(4, 4)

# QR decomposition
U, R = np.linalg.qr(A)

```

```
# Ensure a uniformly distributed orthonormal matrix by adjusting the sign
U *= np.sign(np.linalg.det(Q))

print("Orthonormal matrix Q:")
print(U)
```

Orthonormal matrix Q:

```
[[ 0.37858101  0.59644935  0.23293019 -0.66833235]
 [-0.17753604  0.69361511  0.30885405  0.62608962]
 [ 0.33496798 -0.40212764  0.84294197  0.12465412]
 [-0.84436593 -0.03794228  0.37390043 -0.38184423]]
```

Checking that the basis is orthonormal:

- **Orthogonality:** $\vec{u}_i \vec{u}_j^T = 0$ if $i \neq j$
- **Normality** $\vec{u}_i \vec{u}_i = 1$
- In matrix notation:

$$\mathbf{U}\mathbf{U}^T = \begin{bmatrix} 1, 0, \dots, 0 \\ 0, 1, \dots, 0 \\ \vdots \\ 0, 0, \dots, 1 \end{bmatrix} = \mathbf{I}$$

```
In [33]: # Verify orthonormality: Q^T Q = I
print("\nU^T U:")
print(np.round(U.T @ U, decimals=6))
```

U^T U:

```
[[ 1. -0. -0.  0.]
 [-0.  1. -0.  0.]
 [-0. -0.  1.  0.]
 [ 0.  0.  0.  1.]]
```

The Identity Matrix

The identity matrix behaves like the number 1:

The dot product of any matrix $\mathbf{A}$ by the identity matrix $\mathbf{I}$ yields $\mathbf{A}$.

$$\mathbf{AI} = \mathbf{IA} = \mathbf{A}$$

Changing coordinate system

Suppose we have two orthonormal basis:

- The standard basis
 $\vec{e}_1 = [1, 0, 0, \dots, 0], \vec{e}_2 = [0, 1, 0, \dots, 0], \dots, \vec{e}_d = [0, 0, 0, \dots, 1]$
- Some other orthonormal basis : $\mathbf{U}$

- We have two ways of representing a vector $\vec{v}$ as an array of length d :

1. Using the standard coordinate system:

$$\vec{v}_S = [\vec{v} \cdot \vec{e}_1, \vec{v} \cdot \vec{e}_2, \dots, \vec{v} \cdot \vec{e}_d]$$

2. Using $\mathbf{U}$:

$$\vec{v}_U = [\vec{v} \cdot \vec{u}_1, \vec{v} \cdot \vec{u}_2, \dots, \vec{v} \cdot \vec{u}_d]$$

transformations in matrix notation.

- Suppose $\vec{v}_S, \vec{v}_U$ are represented as column vectors
- Transforming $\vec{v}_S$ to $\vec{v}_U$:

$$\vec{v}_U = \mathbf{U} \vec{v}_S = \begin{bmatrix} \vec{u}_1 \cdot \vec{v} \\ \vec{u}_2 \cdot \vec{v} \\ \vdots \\ \vec{u}_d \cdot \vec{v} \end{bmatrix}$$

- Transforming $\vec{v}_U$ to $\vec{v}_S$:

$$\vec{v}_S = \mathbf{U}^{-1} \vec{v}_U = \mathbf{U}^T \vec{v}_U$$

- $\vec{v}$ is **reconstructed exactly** by summing its projections on all of the basis vectors:

$$\vec{v} = (\vec{v} \cdot \vec{u}_1) \vec{u}_1 + (\vec{v} \cdot \vec{u}_2) \vec{u}_2 + \dots + (\vec{v} \cdot \vec{u}_d) \vec{u}_d$$

- In Matrix Notation: $\mathbf{v} = \mathbf{U} \mathbf{U}^T \mathbf{v}$

- equivalently $\mathbf{U} \mathbf{U}^T = \mathbf{I}$

- $\vec{v}$ is **reconstructed APPROXIMATELY** by summing its projections on the basis vectors:

$$\vec{v}_k = (\vec{v} \cdot \vec{u}_1) \vec{u}_1 + (\vec{v} \cdot \vec{u}_2) \vec{u}_2 + \dots + (\vec{v} \cdot \vec{u}_k) \vec{u}_k$$

- The **residual** for the k -approximation is $\vec{r}_k = \vec{v} - \vec{v}_k$, the **residual error** is the squared norm of $\vec{r}$ which is $\|\vec{r}\|_2^2 = \sum_{i=1}^d r_i^2$

```
In [36]: vs=np.array([1,2,3,4])
         vu=U.T @ vs
         vu
```

```
Out[36]: array([-2.34905088,  0.62552755,  4.87506594, -0.56956768])
```

```
In [37]: U @ vu
```

```
Out [37]: array([1., 2., 3., 4.])
```

```
In [38]: vu[-1]=0
         U@vu
```

```
Out [38]: array([0.6193395 , 2.35660041, 3.07099896, 3.78251387])
```

Inverting a Matrix

Recall that the multiplicative inverse of the number a is $a^{-1} = 1/a$

The property of a^{-1} is that $aa^{-1} = 1$.

Recall also that 0 does not have a multiplicative inverse.

Some square matrices $\mathbf{A}$ have a multiplicative inverse $\mathbf{A}^{-1}$ such that $\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$

Finding the inverse of a matrix is called *inverting* the matrix.

An $n \times n$ matrix $\mathbf{A}$ represents a linear transformation from R^n to R^n . If the matrix is **invertible** then there is another transformation $\mathbf{A}^{-1}$ that represents the inverse transformation, such that for any column vector $\mathbf{v} \in R^n$:

$$\mathbf{A}^{-1}\mathbf{A}\mathbf{v} = \mathbf{A}\mathbf{A}^{-1}\mathbf{v} = \mathbf{v}$$

Inverting a 2X2 matrix

Consider the square 2×2 matrix $\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$. The inverse of matrix $\mathbf{A}$ is

$$\mathbf{A}^{-1} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}^{-1} = \frac{1}{a_{11}a_{22} - a_{12}a_{21}} \begin{bmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{bmatrix} \quad (8)$$

Exercise: Check that $\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$

For more information on inverting matrices, see this

http://en.wikipedia.org/wiki/Matrix_inversion.

```
In [39]: # An example of computing the inverse using numpy.linalg.inv
         # note, we need a square matrix (# rows = # cols), use C:
         C = np.random.randn(2,2)
         print("C=\n",C)
         C_inverse = np.linalg.inv(C)
         print("C_inverse=\n",C_inverse)
```

```

C=
[[ 0.20308121 -0.55866709]
 [-0.47561108  1.02123463]]
C_inverse=
[[-17.51247469  -9.58021113]
 [ -8.1559386   -3.48250482]]

```

Checking that $C \times C^{-1} = I$:

```

In [47]: I = np.eye(2)
print("identity matrix=\n",I)
print("C.dot(C_inverse)=\n",np.round(C.dot(C_inverse),decimals=6))
print("C_inverse.dot(C)=\n",np.round(C_inverse.dot(C),decimals=6))

identity matrix=
[[1. 0.]
 [0. 1.]]
C.dot(C_inverse)=
[[1. 0.]
 [0. 1.]]
C_inverse.dot(C)=
[[1. 0.]
 [0. 1.]]

```

Singular matrices

Not all matrices have an inverse. Those that do not are called **singular**

```

In [ ]: C=np.array([[1,0],[1,0]])
print("C=\n",C)
try:
    C_inverse = np.linalg.inv(C)
except:
    print('C cannot be inverted: it is a singular matrix')

```

Pseudo-Inverse

The inverse matrix for A reverses the mapping A . When A is singular it is impossible to reverse the mapping because some information is lost.

In that case we might settle for a solution that maps $A\vec{v}$ to some vector close to $\vec{v}$. If we are interested in minimizing the square of those distances we can use the **Pseudo-Inverse**

```

In [49]: import numpy as np

# Create an overdetermined system: 3 equations, 2 unknowns
A = np.array([[1, 1],
              [1, 2],
              [1, 3]])

b = np.array([1, 2, 2])

```

```

# Compute the pseudo-inverse of A
A_pinv = np.linalg.pinv(A)

# Use it to compute the least-squares solution:  $x = A^+ b$ 
x = A_pinv @ b

# Predicted output
b_pred = A @ x

# Display results
print("Pseudo-inverse of A:\n", A_pinv)
print("\nLeast-squares solution x:\n", x)
print("\nPredicted b (A @ x):\n", b_pred)
print("\nResidual (b - A @ x):\n", b - b_pred)

```

Pseudo-inverse of A:

```
[[ 1.33333333e+00  3.33333333e-01 -6.66666667e-01]
 [-5.00000000e-01  1.03871725e-16  5.00000000e-01]]
```

Least-squares solution x:

```
[0.66666667 0.5      ]
```

Predicted b (A @ x):

```
[1.16666667 1.66666667 2.16666667]
```

Residual (b - A @ x):

```
[-0.16666667 0.33333333 -0.16666667]
```

The Outer Product

- Dot Product = Inner Product

$$[1, 2, 3] \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} = 1 \times 1 + 2 \times 1 + 3 \times 0 = 3$$

- Outer Product:

$$\begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} [1, 2, 3] = \begin{bmatrix} 1, 2, 3 \\ 1, 2, 3 \\ 0, 0, 0 \end{bmatrix}$$

Next video: solving a set of linear equations

Exercises

1. Write a function that takes as input an array of real numbers $a_1, a_2, \dots, a_d$ and outputs a $d \times d$ "scaling" matrix $\mathbf{S}$ such that for any row vector $\mathbf{v} = [v_1, v_2, \dots, v_d]$

:

$$\mathbf{vS} = [a_1 \times v_1, a_2 \times v_2, \dots, a_d \times v_d]$$

2. Write a function that takes as input an array of real numbers $a_1, a_2, \dots, a_d$ and output the **inverse** of $\mathbf{S}$, denoted $\mathbf{S}^{-1}$
3. Orthonormal transformation are also called **rotations**. In [2D](#), the rotation has a single rotation angle θ :

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

- Write a function that takes θ as it's input and returns the rotation matrix $R(\theta)$
- Write a function that takes θ as it's input and returns the rotation matrix $R^{-1}(\theta)$. (hint: all you need to do is call the previous function with a different parameter).
- Write a function that takes as input an angle θ and a list of points $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$, rotates the points using the matrix $R(\theta)$, plots the line connecting the rotated points, and returns the rotated points themselves.

In []: